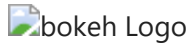
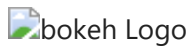


Bokeh for Time Series Analysis



****Introduction to Bokeh**** Bokeh is an interactive visualization library for Python that targets modern web browsers for presentation. Unlike Matplotlib, which is primarily designed for static plots, Bokeh excels at creating interactive plots and dashboards. It can handle large datasets and streaming data, making it suitable for real-time applications.

Key Features of Bokeh:

- **Interactivity:** Built-in support for zooming, panning, hovering, and other interactive tools.
 - **Web-Focused:** Generates HTML and JavaScript, making it easy to embed plots in web pages.
 - **High Performance:** Can handle large datasets efficiently.
 - **Versatility:** Supports a wide range of plot types (lines, bars, scatter plots, etc.).
-

Documentation:

For comprehensive documentation, please refer to the official Bokeh website:

<https://docs.bokeh.org/en/latest/>

Lab Exercise:

Your task is to recreate the time series analysis lab we previously conducted using Pandas, Matplotlib, and Seaborn, but this time, utilize the Bokeh library for visualization. This will involve:

1. Loading and preprocessing the "AirPassengersDates.csv" dataset.
2. Creating interactive Bokeh plots for:
 - Time series line plots
 - Bar plots of aggregated data
 - Visualizing mean and standard deviation
 - Outlier detection
 - Resampling (upsampling and downsampling)
 - Lag analysis
 - Autocorrelation

Pay close attention to Bokeh's features for interactivity (tools, hover effects) and its handling of data sources. Aim to replicate the insights and visualizations from the previous lab while leveraging Bokeh's strengths.

Good luck!

```
In [60]: # %%
# Section 1: Setting Up and Loading Data
# -----

# 1. Import Libraries
# ~~~~~

import pandas as pd
import numpy as np
from pathlib import Path
import bokeh.plotting as bp
from bokeh.io import output_notebook
from bokeh.models import Span, ColumnDataSource

# Set the data path
DATA_PATH = Path(".") # Assuming the dataset is in the same directory

# 2. Load the Dataset
# ~~~~~

passenger_df = pd.read_csv("./datasets/AirPassengersDates.csv")

# Display the first few rows
print(passenger_df.head())

# Show information about the DataFrame
print(passenger_df.info())
```

```
      Date  #Passengers
0  1949-01-01         112
1  1949-02-01         118
2  1949-03-01         132
3  1949-04-01         129
4  1949-05-01         121
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144 entries, 0 to 143
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        144 non-null    object
1   #Passengers  144 non-null    int64
dtypes: int64(1), object(1)
memory usage: 2.4+ KB
None
```

```
In [61]: from bokeh.io import output_notebook

# %%
```

```

# Section 2: Date/Time Manipulation
# -----

# 1. Convert to Datetime
# ~~~~~

passenger_df["Date"] = pd.to_datetime(passenger_df["Date"])

# Verify the conversion
print(passenger_df.info())

# 2. Extract Date Components
# ~~~~~

passenger_df["Month"] = passenger_df["Date"].dt.month
passenger_df["Day"] = passenger_df["Date"].dt.day
passenger_df["Day_Name"] = passenger_df["Date"].dt.day_name()

print(passenger_df.head())

# %%
# Section 3: Time Series Visualization and Analysis
# -----

# 1. Basic Time Series Plot
# ~~~~~

output_notebook()

p = bp.figure(
    title="Air Passengers Over Time",
    x_axis_type="datetime"
)

p.line(
    passenger_df["Date"],
    passenger_df["#Passengers"],
)

p.xaxis.axis_label = "Date"
p.yaxis.axis_label = "Number of Passengers"

bp.show(p)

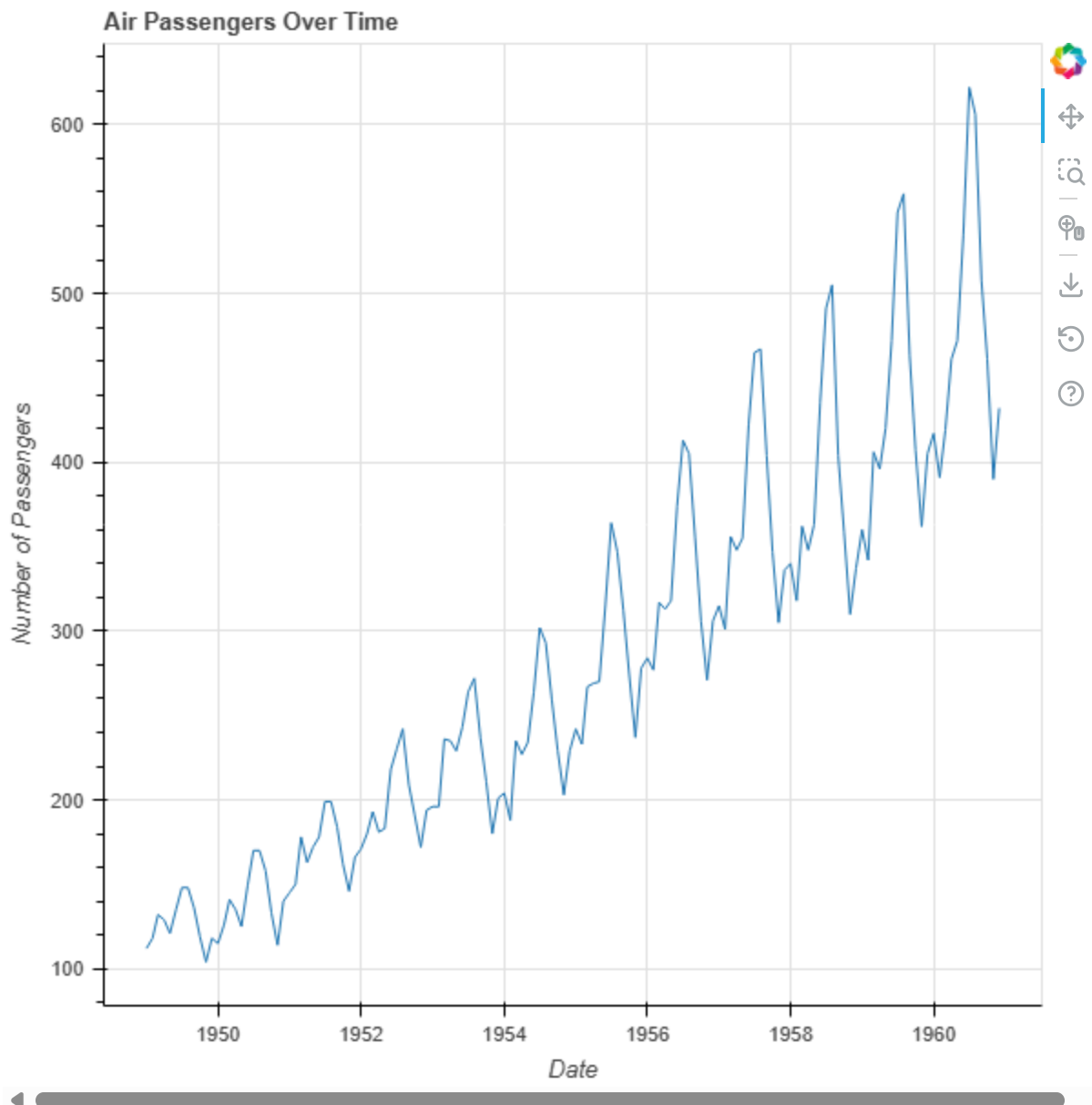
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 144 entries, 0 to 143  
Data columns (total 2 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   Date        144 non-null   datetime64[ns]  
1   #Passengers  144 non-null   int64  
dtypes: datetime64[ns](1), int64(1)  
memory usage: 2.4 KB  
None
```

	Date	#Passengers	Month	Day	Day_Name
0	1949-01-01	112	1	1	Saturday
1	1949-02-01	118	2	1	Tuesday
2	1949-03-01	132	3	1	Tuesday
3	1949-04-01	129	4	1	Friday
4	1949-05-01	121	5	1	Sunday



BokehJS 3.8.2 successfully loaded.



```
In [62]: # 2. Aggregation and Bar Plot
# ~~~~~

passengers_per_month = (
    passenger_df.groupby("Month")["#Passengers"].sum().reset_index()
)

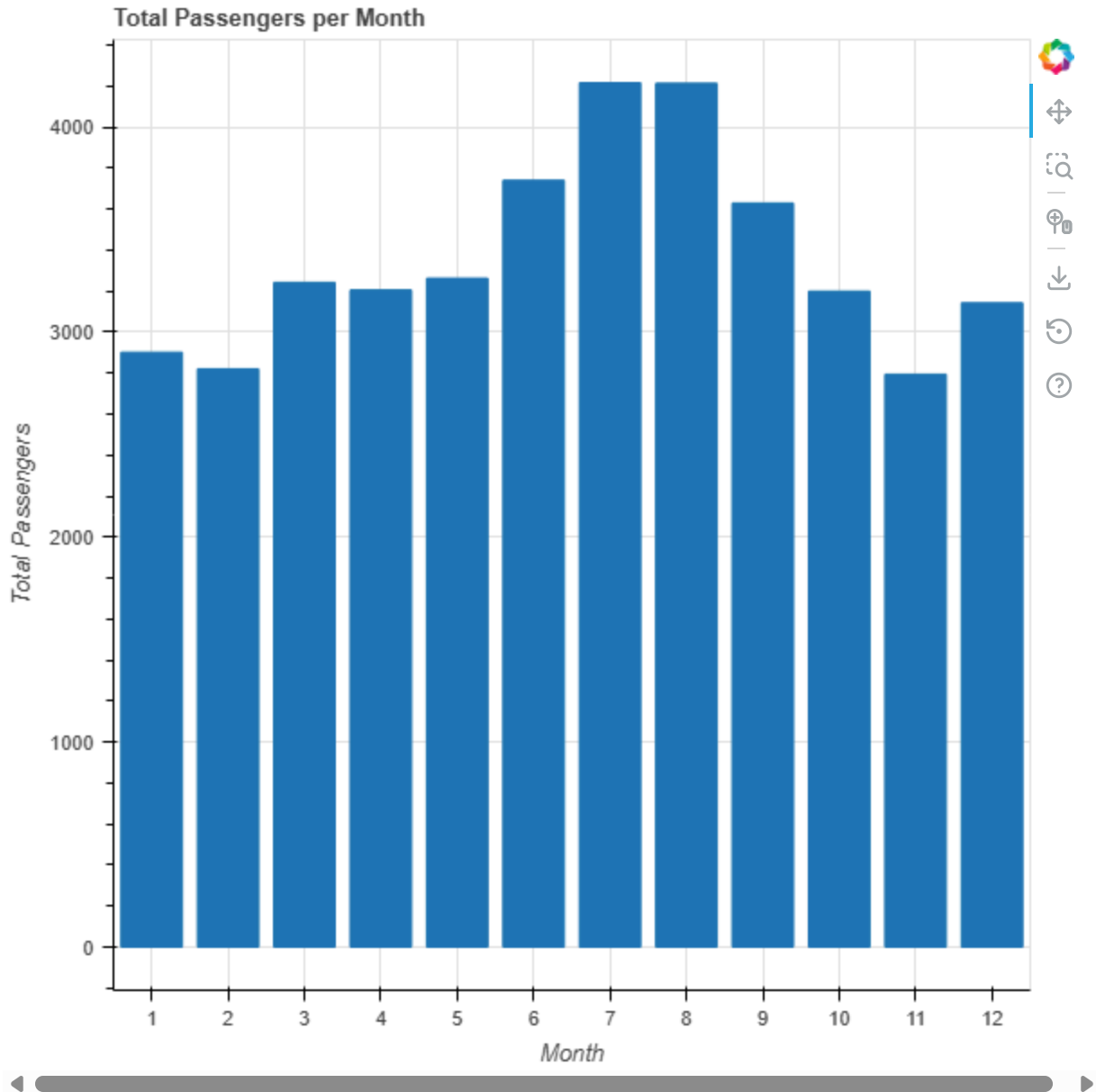
months = [str(m) for m in passengers_per_month["Month"]]
source = ColumnDataSource(data=dict(
    months=months,
    passengers=passengers_per_month["#Passengers"]
))

p = bp.figure(
    title="Total Passengers per Month",
    x_range=months,
)

p.vbar(x="months", top="passengers", width=0.8, source=source)

p.xaxis.axis_label = "Month"
p.yaxis.axis_label = "Total Passengers"

bp.show(p)
```



```
In [63]: # 3. Mean and Standard Deviation
# ~~~~~

mean_passengers = passenger_df["#Passengers"].mean()
std_passengers = passenger_df["#Passengers"].std()

p = bp.figure(
    title="Passengers with Mean and Standard Deviation",
    x_axis_type="datetime",
)

p.line(passenger_df["Date"], passenger_df["#Passengers"], legend_label="Passengers")

# Mean Line
mean_line = Span(location=mean_passengers, dimension="width", line_color="red", lin
p.add_layout(mean_line)

# Mean + Std Line
upper_line = Span(location=mean_passengers + std_passengers, dimension="width", lin
```

```

p.add_layout(upper_line)

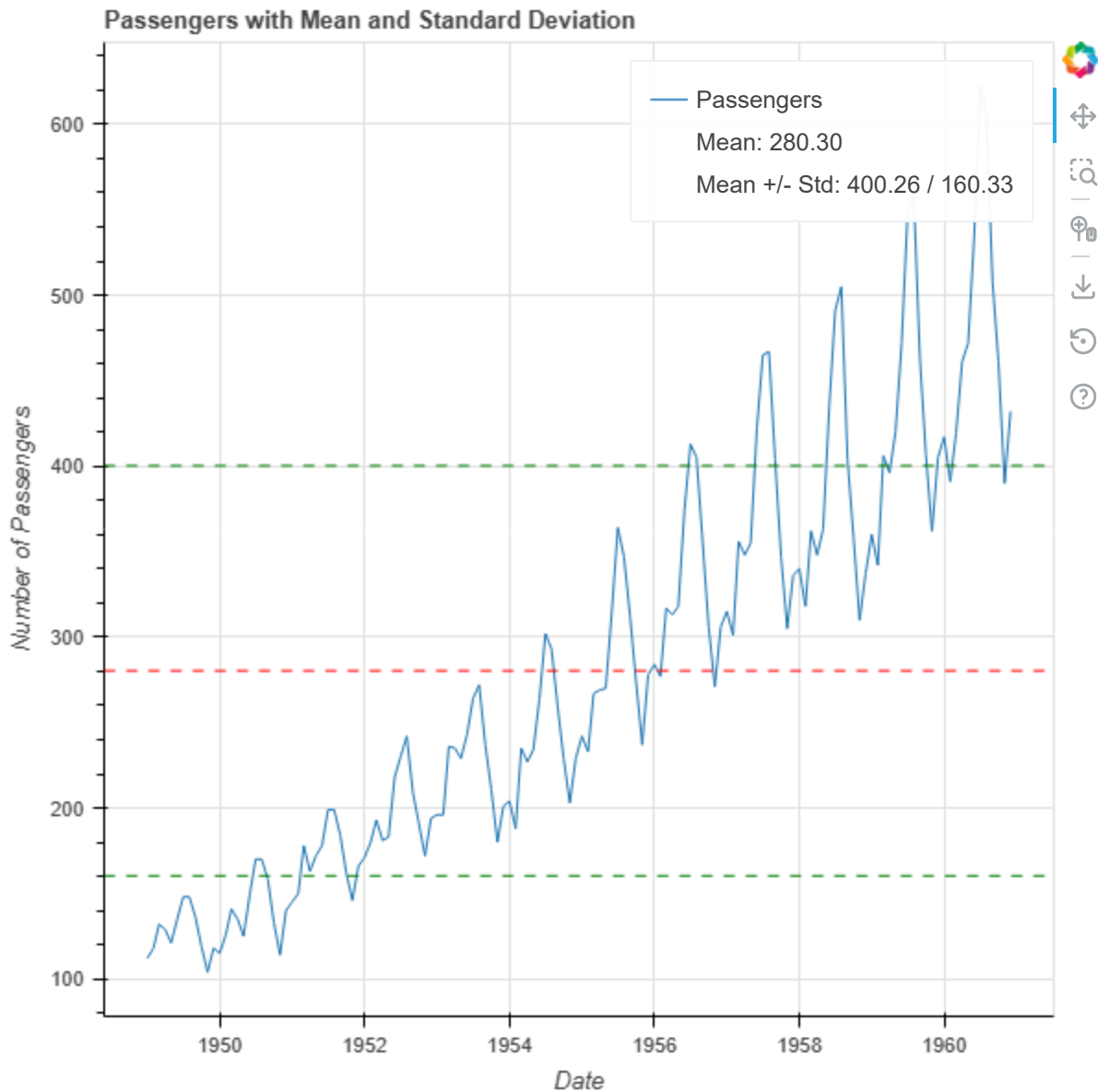
# Mean - Std Line
lower_line = Span(location=mean_passengers - std_passengers, dimension="width", lin
p.add_layout(lower_line)

# Add invisible lines for legend entries
p.line([], [], line_color="red", line_dash="dashed", legend_label=f"Mean: {mean_pas
p.line([], [], line_color="green", line_dash="dashed", legend_label=f"Mean +/- Std:

p.xaxis.axis_label = "Date"
p.yaxis.axis_label = "Number of Passengers"

bp.show(p)

```



```

In [64]: # %%
# Section 4: Outlier Detection
# -----

```

```
# 1. Z-Score Calculation
# ~~~~~

passenger_df["Z-Score"] = (
    passenger_df["#Passengers"] - passenger_df["#Passengers"].mean()
) / passenger_df["#Passengers"].std()
passenger_df["Absolute_Z-Score"] = abs(passenger_df["Z-Score"])

print(passenger_df.sort_values(by="Absolute_Z-Score", ascending=False).head(10))
```

	Date	#Passengers	Month	Day	Day_Name	Z-Score	Absolute_Z-Score
138	1960-07-01	622	7	1	Friday	2.848311	2.848311
139	1960-08-01	606	8	1	Monday	2.714940	2.714940
127	1959-08-01	559	8	1	Saturday	2.323164	2.323164
126	1959-07-01	548	7	1	Wednesday	2.231471	2.231471
137	1960-06-01	535	6	1	Wednesday	2.123108	2.123108
140	1960-09-01	508	9	1	Thursday	1.898044	1.898044
115	1958-08-01	505	8	1	Friday	1.873037	1.873037
114	1958-07-01	491	7	1	Tuesday	1.756338	1.756338
136	1960-05-01	472	5	1	Sunday	1.597960	1.597960
125	1959-06-01	472	6	1	Monday	1.597960	1.597960

```
In [65]: # 2. Outlier Visualization
# ~~~~~

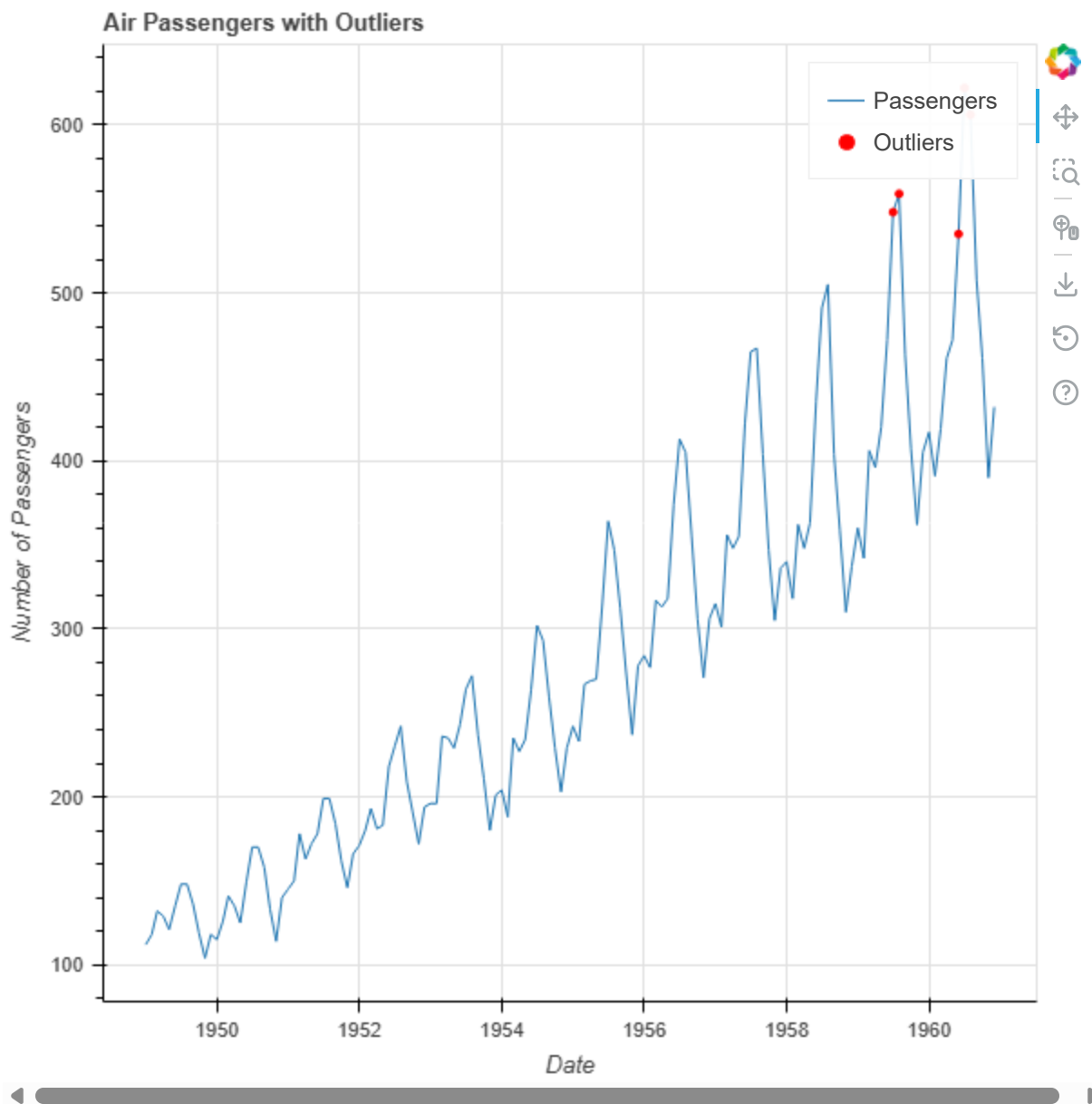
outliers = passenger_df[(passenger_df["Absolute_Z-Score"] > 2)] # Define outliers

p = bp.figure(
    title="Air Passengers with Outliers",
    x_axis_type="datetime",
)

p.line(passenger_df["Date"], passenger_df["#Passengers"], legend_label="Passengers")
p.scatter(outliers["Date"], outliers["#Passengers"], color="red", legend_label="Outliers")

p.xaxis.axis_label = "Date"
p.yaxis.axis_label = "Number of Passengers"

bp.show(p)
```

```
In [66]: # %%
# Section 5: Resampling
# -----

passenger_df.set_index(
    "Date", inplace=True
) # Set 'Date' as index for resampling

# 1. Upsampling
# ~~~~~

# Upsample to daily frequency
daily_passengers = passenger_df.resample('D').asfreq()

# Interpolate missing values
daily_passengers['#Passengers'] = daily_passengers['#Passengers'].interpolate(metho

p = bp.figure(
    title="Upsampling to Daily Frequency",
```

```

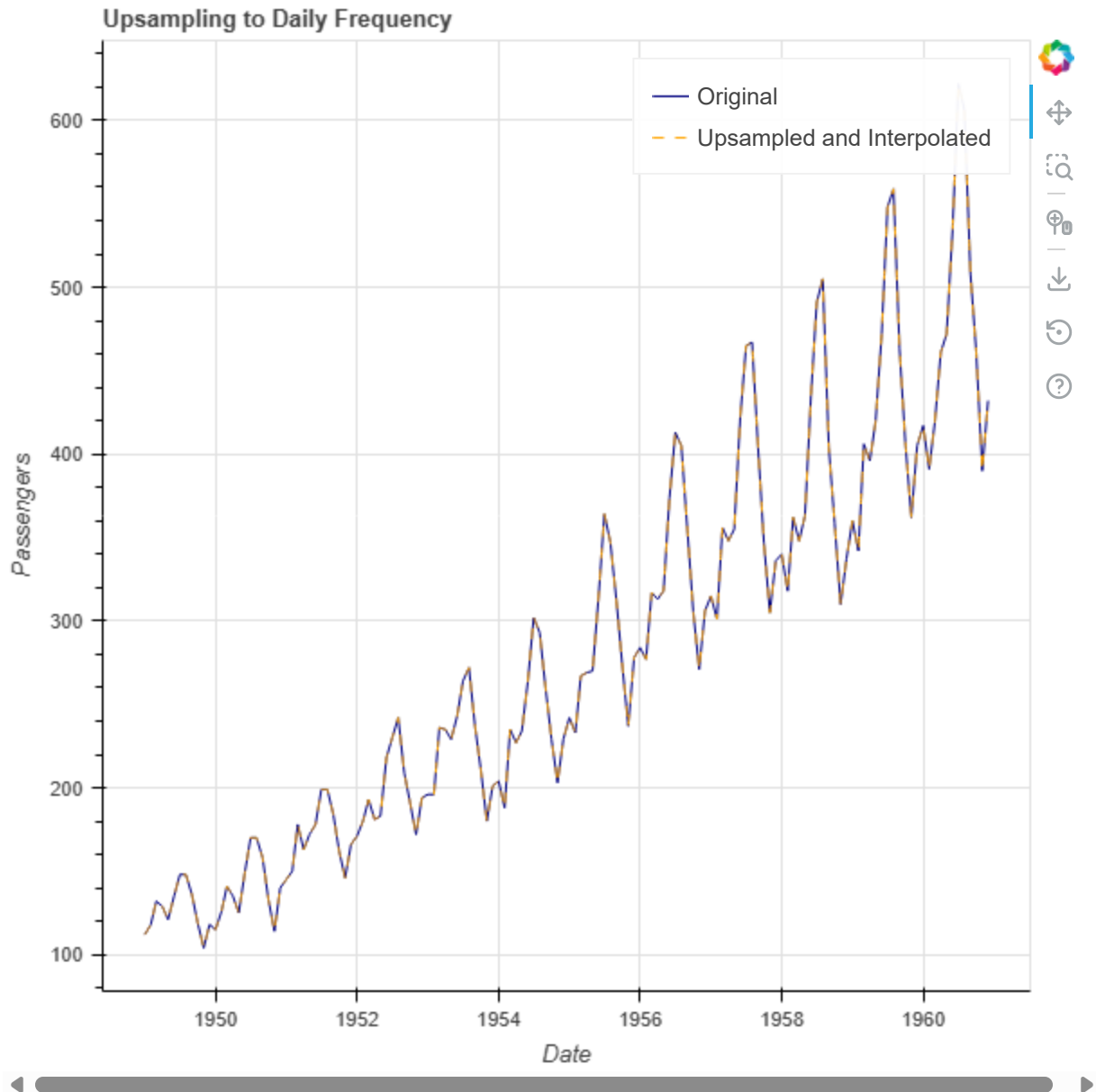
x_axis_type="datetime",
)

p.line(passenger_df.index, passenger_df['#Passengers'], color="navy", legend_label="Original")
p.line(daily_passengers.index, daily_passengers['#Passengers'], line_dash="dashed", color="yellow", legend_label="Upsampled and Interpolated")

p.xaxis.axis_label = "Date"
p.yaxis.axis_label = "Passengers"

bp.show(p)

```



```

In [67]: # 2. Downsampling
# ~~~~~

# Downsample to yearly frequency
yearly_passengers = passenger_df.resample("Y")["#Passengers"].mean()

p = bp.figure(
    title="Downsampling to Yearly Frequency",

```

```

x_axis_type="datetime",
)

p.line(passenger_df.index, passenger_df["#Passengers"], color="navy", legend_label="Original")
p.line(yearly_passengers.index, yearly_passengers.values, color="orange", legend_label="Yearly Average")
p.scatter(yearly_passengers.index, yearly_passengers.values, color="orange")

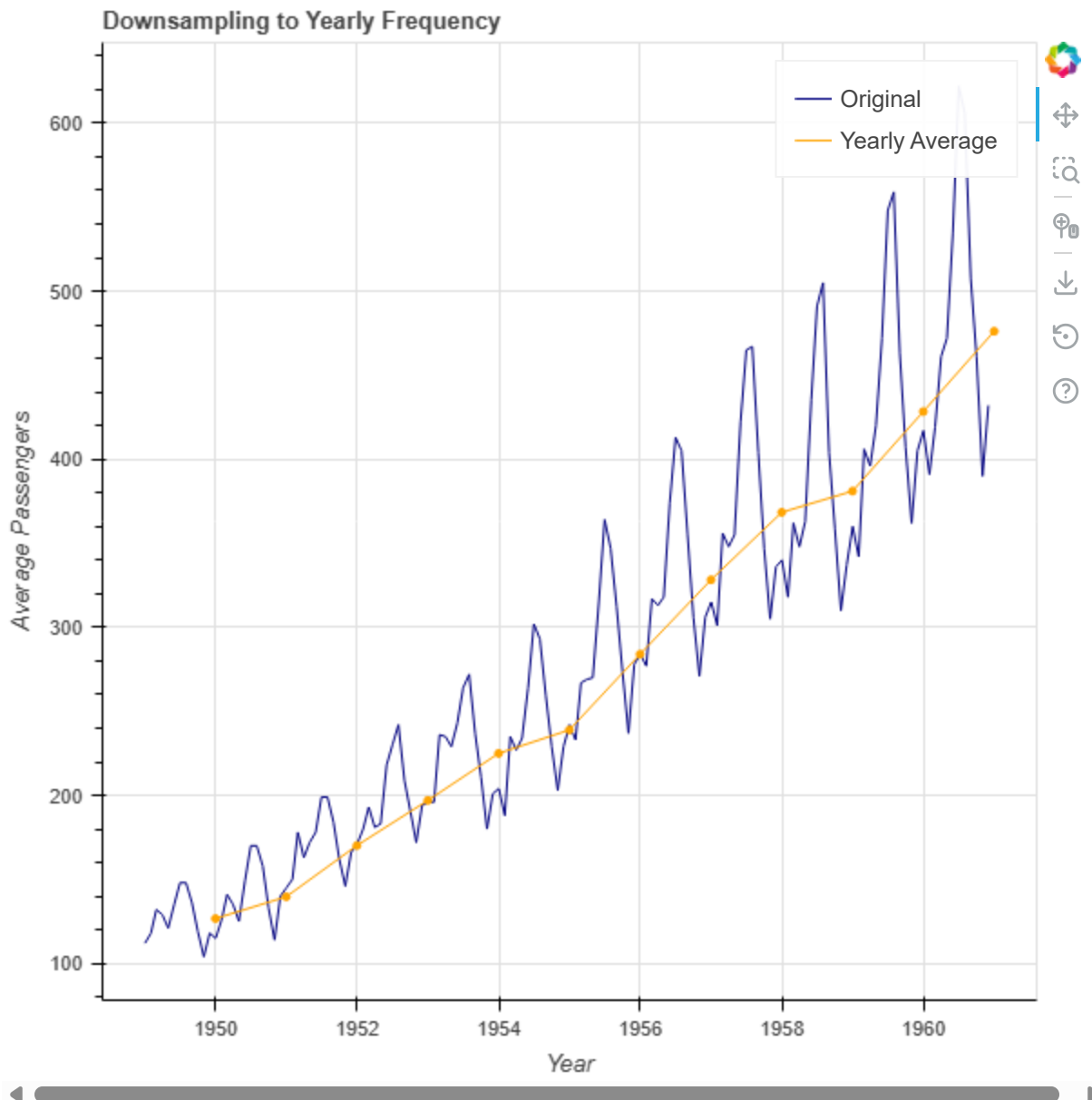
p.xaxis.axis_label = "Year"
p.yaxis.axis_label = "Average Passengers"

bp.show(p)

```

C:\Users\naeld\AppData\Local\Temp\ipykernel_31264\671064778.py:5: FutureWarning: 'Y' is deprecated and will be removed in a future version, please use 'YE' instead.

```
yearly_passengers = passenger_df.resample("Y")["#Passengers"].mean()
```



```

In [68]: # %%
# Section 6: Shift and tshift (Lag Analysis)
# -----

```

```

# 1. Using `shift()` for Lag Analysis
# ~~~~~

# Ensure 'Date' is the index (DatetimeIndex)
#passenger_df.set_index("Date", inplace=True)

passenger_df.reset_index(inplace=False) # Reset index to use Date as column

passenger_df["#Passengers_Shift"] = passenger_df["#Passengers"].shift(
    periods=1
) # Shift data by 1 period
passenger_df["#Passengers_tShift"] = passenger_df["#Passengers"].shift(
    periods=1, freq="MS"
) # Shift index by 1 month start

print(passenger_df.head())

# Visualization
p = bp.figure(
    title="Shift vs tShift",
    x_axis_type="datetime",
)

p.line(passenger_df.index, passenger_df["#Passengers"], color="navy", legend_label=
p.line(passenger_df.index, passenger_df["#Passengers_Shift"], color="orange", legen
p.line(passenger_df["#Passengers_tShift"].index, passenger_df["#Passengers_tShift"]

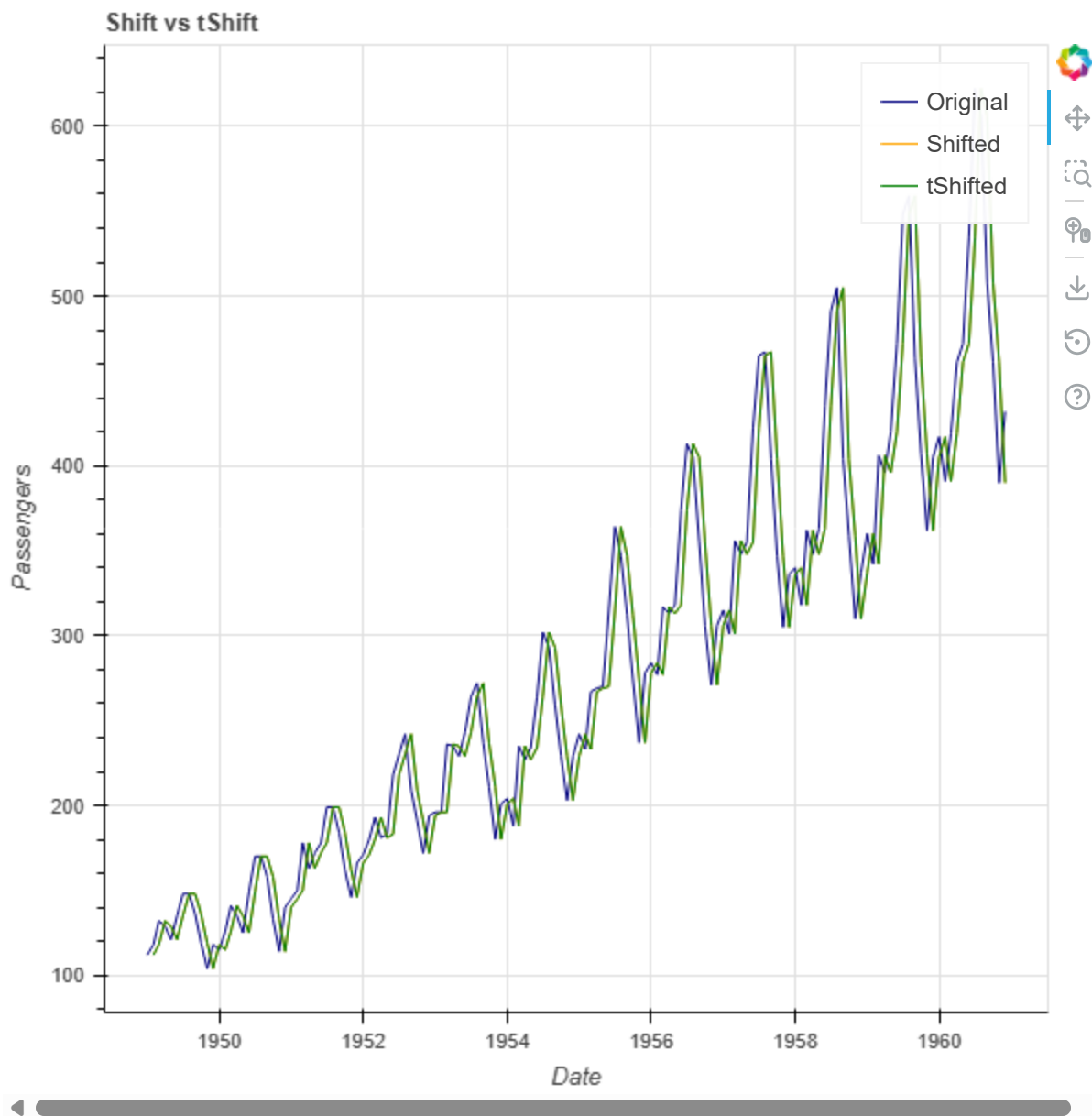
p.xaxis.axis_label = "Date"
p.yaxis.axis_label = "Passengers"

bp.show(p)

```

	#Passengers	Month	Day	Day_Name	Z-Score	Absolute_Z-Score	\
Date							
1949-01-01	112	1	1	Saturday	-1.402882	1.402882	
1949-02-01	118	2	1	Tuesday	-1.352868	1.352868	
1949-03-01	132	3	1	Tuesday	-1.236169	1.236169	
1949-04-01	129	4	1	Friday	-1.261176	1.261176	
1949-05-01	121	5	1	Sunday	-1.327861	1.327861	

	#Passengers_Shift	#Passengers_tShift
Date		
1949-01-01	NaN	NaN
1949-02-01	112.0	112.0
1949-03-01	118.0	118.0
1949-04-01	132.0	132.0
1949-05-01	129.0	129.0



```
In [69]: # %%
# Section 7: Autocorrelation
# -----

# 1. Autocorrelation Plot
# ~~~~~

from statsmodels.tsa.stattools import acf

acf_values = acf(passenger_df["#Passengers"].dropna(), nlags=30)
lags = list(range(len(acf_values)))

p = bp.figure(
    title="Autocorrelation Function (ACF)",
)

# Confidence interval (approximate 95%)
n = len(passenger_df["#Passengers"].dropna())
conf = 1.96 / np.sqrt(n)
```

```

conf_band = Span(location=conf, dimension="width", line_dash="dashed")
neg_conf_band = Span(location=-conf, dimension="width", line_dash="dashed")
p.add_layout(conf_band)
p.add_layout(neg_conf_band)

# Stem-like plot: vertical lines + circles
for lag, val in zip(lags, acf_values):
    p.segment(x0=lag, y0=0, x1=lag, y1=val)

p.scatter(lags, acf_values)

p.xaxis.axis_label = "Lag"
p.yaxis.axis_label = "Autocorrelation"

bp.show(p)

```

